09 — Slow Query Analysis

> *"You can't optimize what you can't measure. Find the slow queries first."*

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Set up pg_stat_statements and slow query logging
- Find the top slow, most-called, and highest-impact queries
- Distinguish I/O-bound from CPU-bound query bottlenecks
- Use pg_stat_activity to diagnose live query problems
- Set up auto_explain for automatic plan capture
- Build a repeatable slow query investigation workflow

---

pg_stat_statements: The Foundation

`pg_stat_statements` is the essential extension for slow query analysis. It tracks every query's execution statistics across all sessions.

Setup

```
-- Enable in postgresql.conf:
-- shared_preload_libraries = 'pg_stat_statements'
-- pg_stat_statements.track = all
```

```
-- pg_stat_statements.max = 5000

-- Create extension (after server restart):
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Verify it's running:
SELECT count(*) FROM pg_stat_statements;
```

Key Columns

Column	Description
`query`	Normalized query text (parameters replaced with $1, $2...)
`calls`	Number of times this query was executed
`total_exec_time`	Total execution time (ms) across all calls
`mean_exec_time`	Average execution time per call (ms)
`min_exec_time`	Minimum execution time
`max_exec_time`	Maximum execution time
`stddev_exec_time`	Standard deviation of execution time
`rows`	Total rows returned/affected
`shared_blks_hit`	Total shared buffer hits
`shared_blks_read`	Total pages read from disk
`blk_read_time`	Time spent reading blocks from disk (ms)
`blk_write_time`	Time spent writing blocks (ms)
`temp_blks_read`	Temp file pages read (spill)
`temp_blks_written`	Temp file pages written (spill)
`wal_bytes`	WAL bytes generated

PostgreSQL 14+ Additions

```
-- pg_stat_statements in PG14+ also tracks:
-- total_plan_time, mean_plan_time (planning overhead)
-- jit_functions, jit_optimization_count (JIT compilation)
```

Setting Up Slow Query Logging

Configure PostgreSQL to log slow queries to the server log:

```
-- In postgresql.conf (or ALTER SYSTEM):
ALTER SYSTEM SET log_min_duration_statement = 1000;   -- log queries > 1 second
ALTER SYSTEM SET log_min_duration_statement = 500;    -- log queries > 500ms
ALTER SYSTEM SET log_min_duration_statement = -1;     -- disable slow query log

-- Log all queries (WARNING: extremely verbose in production)
ALTER SYSTEM SET log_min_duration_statement = 0;

-- Log statement type along with duration:
ALTER SYSTEM SET log_statement = 'none';              -- don't log all statements
ALTER SYSTEM SET log_duration = off;                  -- don't log ALL durations

-- Apply changes:
SELECT pg_reload_conf();
```

Log Format Configuration

```
-- Add useful fields to log messages:
ALTER SYSTEM SET log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h
';
-- Fields: timestamp, pid, session line, user, database, application, client IP

-- Log query parameters (careful: may contain sensitive data):
ALTER SYSTEM SET log_parameter_max_length = 1024;     -- log first 1024 chars of
parameters
ALTER SYSTEM SET log_parameter_max_length_on_error = 64;  -- for error messages
```

pgBadger: Log File Analysis

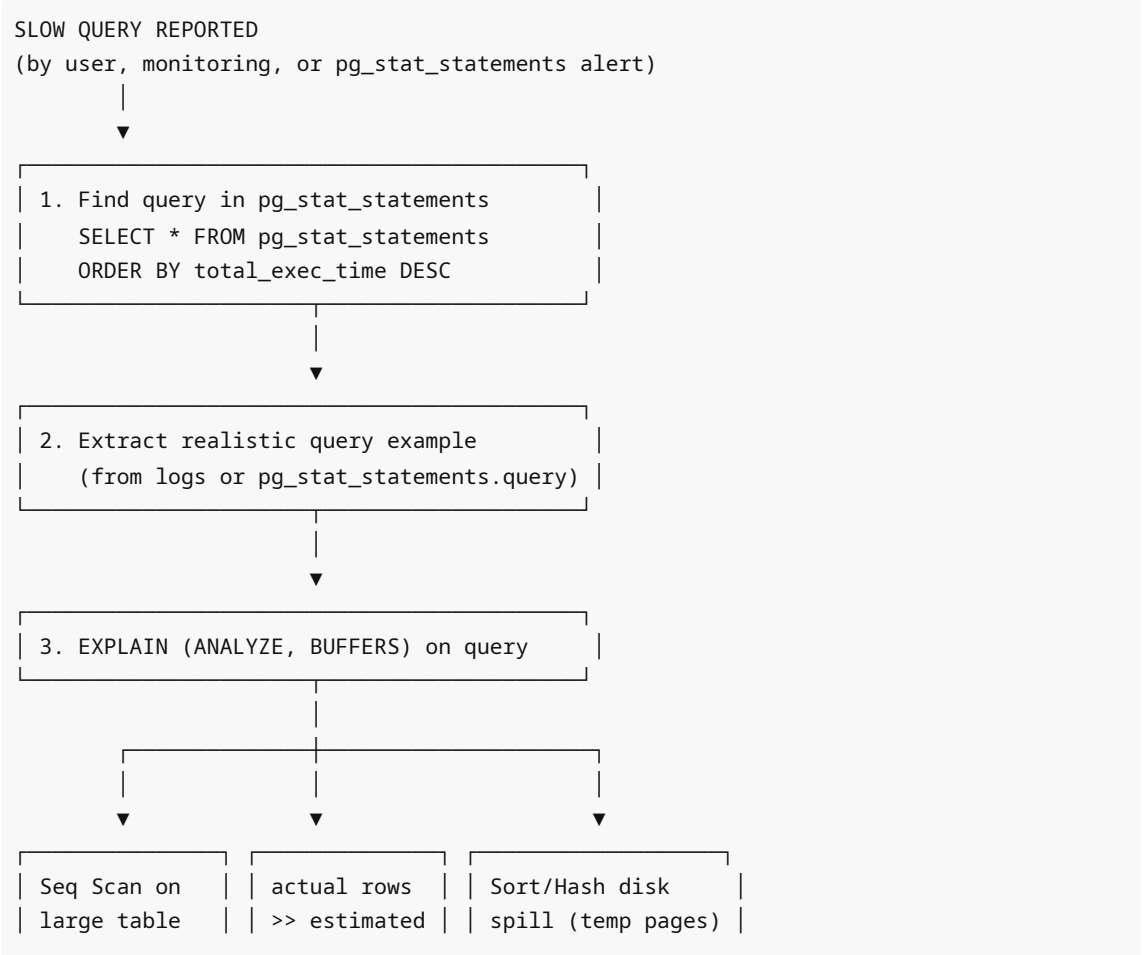
```
# Install pgBadger for HTML reports from PostgreSQL logs:
pgbadger /var/log/postgresql/postgresql-2024-06-15.log -o report.html
# Generates: top slow queries, most frequent queries, lock waits, etc.
```

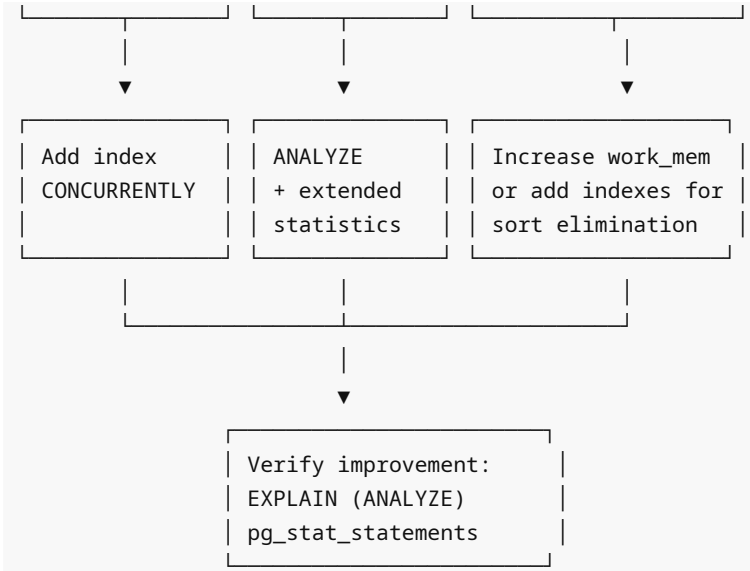
Systematic Analysis Workflow

```
Step 1: MEASURE
  → Enable pg_stat_statements
  → Wait for representative sample period (1 hour minimum, 1 day preferred)
  → Or: reproduce the problem scenario

Step 2: FIND
  → Query pg_stat_statements for top slow/frequent queries
  → Sort by total_exec_time DESC (highest overall impact)
  → Sort by mean_exec_time DESC (slowest per call)
  → Sort by calls DESC (most frequent)

Step 3: ISOLATE
```

```
  → Extract a specific slow query
  → Get a realistic example (real parameter values from logs)
  → Run EXPLAIN (ANALYZE, BUFFERS) on that query

Step 4: DIAGNOSE
  → Check estimated vs actual rows (statistics issue?)
  → Check for Seq Scan on large table (missing index?)
  → Check for disk spill (Sort/Hash Batches, temp buffers)
  → Check for loops × time (Nested Loop on large table?)

Step 5: FIX
  → Add index CONCURRENTLY
  → Rewrite query
  → Update statistics with ANALYZE
  → Tune work_mem for this query

Step 6: VERIFY
  → Repeat EXPLAIN (ANALYZE, BUFFERS)
  → Compare pg_stat_statements before/after
  → Monitor pg_stat_statements for regression
```

ASCII Diagram: Slow Query Investigation Flow





Finding Top Slow Queries

By Total Impact (most important for production optimization)

```sql
-- Queries with highest total execution time (= most CPU/IO consumed)
SELECT
    LEFT(query, 120) AS query_snippet,
    calls,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms,
    ROUND(stddev_exec_time::numeric, 2) AS stddev_ms,
    rows,
    ROUND(shared_blks_read * 100.0 / NULLIF(shared_blks_hit + shared_blks_read, 0),
1)
        AS cache_miss_pct
FROM pg_stat_statements
WHERE calls > 10
  AND total_exec_time > 1000  -- at least 1 second total
ORDER BY total_exec_time DESC
LIMIT 20;
```

By Slowest Average Execution

```sql
-- Queries with highest average execution time per call
SELECT
    LEFT(query, 120) AS query_snippet,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms,
    ROUND(max_exec_time::numeric, 2) AS max_ms,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec
FROM pg_stat_statements
```

```
WHERE calls > 5
ORDER BY mean_exec_time DESC
LIMIT 20;
```

By Most Calls (N+1 detection)

```
-- Most frequently called queries (potential N+1 pattern)
SELECT
    LEFT(query, 120) AS query_snippet,
    calls,
    ROUND(mean_exec_time::numeric, 3) AS mean_ms,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec
FROM pg_stat_statements
ORDER BY calls DESC
LIMIT 20;
-- If a simple SELECT by ID is called millions of times per day:
-- → Possible N+1 or loop without batching
```

By I/O (Disk Reads)

```
-- Queries causing the most disk I/O
SELECT
    LEFT(query, 120) AS query_snippet,
    calls,
    shared_blks_read AS disk_pages_read,
    ROUND(blk_read_time::numeric, 2) AS disk_read_ms,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms
FROM pg_stat_statements
WHERE shared_blks_read > 1000
ORDER BY shared_blks_read DESC
LIMIT 20;
```

By Temp File Usage (Memory Spill)

```
-- Queries spilling to temp files (work_mem too low)
SELECT
    LEFT(query, 120) AS query_snippet,
    calls,
    temp_blks_read,
    temp_blks_written,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms
FROM pg_stat_statements
WHERE temp_blks_read > 0 OR temp_blks_written > 0
ORDER BY temp_blks_written DESC
LIMIT 20;
```

Identifying High-Variance Queries

Queries with high variance (stddev >> mean) are inconsistent — sometimes fast, sometimes slow. These are often victims of lock contention or cache effects.

```
-- High variance queries (inconsistent performance)
SELECT
    LEFT(query, 120) AS query_snippet,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms,
    ROUND(stddev_exec_time::numeric, 2) AS stddev_ms,
    ROUND(max_exec_time::numeric, 2) AS max_ms,
    ROUND(stddev_exec_time / NULLIF(mean_exec_time, 0), 2) AS cv  -- coefficient of
variation
FROM pg_stat_statements
WHERE calls > 50
  AND stddev_exec_time > mean_exec_time  -- stddev > mean = very inconsistent
ORDER BY cv DESC
LIMIT 20;
```

High CV (> 1.0) suggests: lock waits, cache misses on first run, or variable data volume.

Lock Wait Analysis

Slow queries are sometimes slow not from computation but from waiting for locks.

```
-- Queries currently waiting for locks:
SELECT
    pid,
    now() - query_start AS wait_time,
    state,
    wait_event_type,
    wait_event,
    LEFT(query, 100) AS query
FROM pg_stat_activity
WHERE wait_event_type = 'Lock'
ORDER BY wait_time DESC;

-- Find what holds the lock that's blocking:
SELECT
    blocked.pid AS blocked_pid,
    blocked.query AS blocked_query,
    blocking.pid AS blocking_pid,
    blocking.query AS blocking_query,
    blocking.state AS blocking_state
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
    ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE blocked.wait_event_type = 'Lock';
```

---

pg_stat_activity: Live Query Monitoring

```
-- See all currently running queries with wait information
SELECT
    pid,
    usename,
    application_name,
    client_addr,
    state,
    wait_event_type,
    wait_event,
    now() - query_start AS query_age,
    now() - state_change AS state_age,
    LEFT(query, 100) AS query
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start;

-- Queries running longer than 30 seconds:
SELECT
    pid,
    now() - query_start AS duration,
    state,
    LEFT(query, 100) AS query
FROM pg_stat_activity
WHERE state = 'active'
  AND now() - query_start > INTERVAL '30 seconds'
ORDER BY query_start;

-- Cancel a runaway query (graceful):
SELECT pg_cancel_backend(pid);

-- Kill a connection (force):
SELECT pg_terminate_backend(pid);
```

---

auto_explain: Automatic Plan Logging

`auto_explain` logs query plans for queries exceeding a duration threshold.

Setup

```
-- In postgresql.conf (requires restart):
shared_preload_libraries = 'auto_explain'

-- Configure (can do without restart via ALTER SYSTEM):
ALTER SYSTEM SET auto_explain.log_min_duration = 1000;  -- log plans > 1 second
ALTER SYSTEM SET auto_explain.log_analyze = on;          -- include actual
rows/timing
ALTER SYSTEM SET auto_explain.log_buffers = on;          -- include buffer stats
```

```
ALTER SYSTEM SET auto_explain.log_nested_statements = on; -- include nested queries
ALTER SYSTEM SET auto_explain.log_format = 'text';       -- or 'json' for tooling
ALTER SYSTEM SET auto_explain.sample_rate = 0.1;         -- PG12+: log 10% of slow
queries
SELECT pg_reload_conf();
```

Session-Level auto_explain (No Restart)

```
-- Load for current session only:
LOAD 'auto_explain';
SET auto_explain.log_min_duration = 0;  -- log ALL queries for this session
SET auto_explain.log_analyze = on;
-- Run your queries; plans appear in the PostgreSQL log
```

Reading auto_explain Output (from server log)

```
2024-06-15 10:23:45.123 UTC [12345] LOG:  duration: 1234.567 ms  plan:
Query Text: SELECT * FROM orders WHERE customer_id = 42 ORDER BY created_at DESC LIMIT
10
Index Scan using idx_orders_customer on orders  (cost=... actual time=...)
  Index Cond: (customer_id = 42)
  Rows Removed by Filter: 0
  Buffers: shared hit=5
```

Identifying Bottleneck Types

I/O-Bound Queries

Signs in EXPLAIN ANALYZE:

- High `shared_blks_read` (disk reads)
- High `blk_read_time` in pg_stat_statements
- Seq Scan on large tables
- Many random heap fetches from Index Scan

Fixes:

- Add appropriate indexes
- Increase `shared_buffers` and `effective_cache_size`
- Ensure hot data fits in cache
- Use partial or covering indexes to reduce I/O

CPU-Bound Queries

Signs in EXPLAIN ANALYZE:

- High `actual time` but low `shared_blks_read`
- Expensive functions (regex, complex expressions) in Filter
- Large sorts and aggregations
- High `cpu_operator_cost` operations

Fixes:

- Add indexes to avoid scanning and sorting
- Precompute expensive calculations (materialized columns)
- Enable parallel query
- Use JIT compilation for long-running analytical queries

Memory-Bound Queries (work_mem Insufficient)

Signs in EXPLAIN ANALYZE:

- `Sort Method: external merge Disk: NkB`
- `Hash Batches: N > 1`
- `temp_blks_read/written > 0`

Fixes:

- Increase `work_mem` for session: `SET LOCAL work_mem = '256MB'`
- Add indexes to eliminate sort operations
- Break complex queries into steps with temp tables

Common Diagnosis Queries

Complete Slow Query Dashboard

```
-- Master slow query view (run periodically)
SELECT
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms,
    ROUND(stddev_exec_time::numeric, 2) AS stddev_ms,
    ROUND(rows::numeric / calls, 0) AS avg_rows,
    ROUND(100.0 * shared_blks_read / NULLIF(shared_blks_hit + shared_blks_read, 0),
1)
        AS cache_miss_pct,
    temp_blks_written,
    LEFT(query, 100) AS query
FROM pg_stat_statements
WHERE calls > 5
  AND mean_exec_time > 100  -- only queries > 100ms avg
ORDER BY total_exec_time DESC
LIMIT 30;

-- Reset stats for fresh measurement period
-- SELECT pg_stat_statements_reset();  -- WARNING: loses all history!
```

Table-Level I/O Analysis

```
-- Which tables are causing the most I/O?
SELECT
    relname,
    seq_scan,
    seq_tup_read,
```

```
    idx_scan,
    idx_tup_fetch,
    n_dead_tup,
    pg_size_pretty(pg_relation_size(relid)) AS size,
    last_vacuum,
    last_analyze
FROM pg_stat_user_tables
ORDER BY seq_tup_read DESC
LIMIT 20;
```

Tracking Query Regressions

A query regression is when a previously fast query becomes slow. Common causes:

- Data growth (table grew, selectivity changed)
- Schema change (index dropped accidentally)
- Statistics became stale after large data change
- PostgreSQL version upgrade (planner behavior changed)

```
-- Detect regressions: compare pg_stat_statements snapshots
-- Requires saving snapshots to a separate table:

CREATE TABLE query_stats_snapshot AS
SELECT
    md5(query) AS query_hash,
    LEFT(query, 200) AS query_text,
    calls,
    mean_exec_time,
    total_exec_time,
    now() AS snapshot_time
FROM pg_stat_statements;

-- Compare current vs snapshot (after N days):
SELECT
    c.query_text,
    s.mean_exec_time AS mean_before,
    c.mean_exec_time AS mean_current,
    ROUND((c.mean_exec_time / NULLIF(s.mean_exec_time, 0) - 1) * 100, 1) AS
pct_change
FROM pg_stat_statements c
JOIN query_stats_snapshot s ON md5(c.query) = s.query_hash
WHERE c.mean_exec_time > s.mean_exec_time * 1.5  -- 50% slower
  AND c.calls > 100
ORDER BY pct_change DESC
LIMIT 20;
```

Common Mistakes

1. Not having pg_stat_statements enabled

This is the single most common mistake for PostgreSQL performance analysis. It should be enabled by default in all production installations.

2. Only looking at mean_exec_time (ignoring total impact)

A query running in 10ms × 1,000,000 calls = 10,000 seconds total impact. A query running in 10 seconds × 10 calls = 100 seconds total. The 10ms query has 100× more total impact!

3. Analyzing from a "cold" pg_stat_statements

After `pg_stat_statements_reset()` or server restart, wait at least 1 hour (ideally 24h) before analyzing — you need a representative sample.

4. Not capturing the actual query parameters

pg_stat_statements normalizes queries (replaces literals with $1, $2...). To see actual parameters, check server logs with `log_min_duration_statement` configured.

5. Forgetting to check for lock waits

A "slow query" that runs in 0.1ms when tested but 30 seconds in production is usually waiting for a lock, not computationally slow.

Best Practices

1. **Enable pg_stat_statements** on all PostgreSQL instances (small performance overhead, huge diagnostic value).
2. **Set `log_min_duration_statement = 1000`** (1 second) to capture slow queries in logs.
3. **Create a monitoring snapshot schedule**: save pg_stat_statements to a table daily for regression detection.
4. **Use total_exec_time for triage**, not mean_exec_time — focus on what's consuming the most resources overall.
5. **Enable auto_explain** with a reasonable threshold (5-10 seconds) to capture plans for outlier slow queries automatically.
6. **Track `temp_blks_written`** — any non-zero value indicates work_mem should be increased.
7. **Correlate pg_stat_statements with pg_stat_user_tables** — high seq_scan on a large table with matching query in pg_stat_statements = clear index opportunity.

Interview Questions & Answers

Q1. How would you find the slowest queries in a PostgreSQL database?

A: I would query `pg_stat_statements` (after ensuring the extension is enabled):

```
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 20;
```

I'd look at both `total_exec_time DESC` (highest overall impact on the server) and `mean_exec_time DESC` (slowest individual calls). For the top candidates, I'd run `EXPLAIN (ANALYZE, BUFFERS)` with realistic parameter values to diagnose the root cause.

Q2. What is the difference between analyzing by mean_exec_time vs total_exec_time?

A: `mean_exec_time` sorts by the average time per call — useful for finding the slowest individual queries. `total_exec_time` sorts by cumulative time consumed — better for finding the queries with the biggest impact on the overall server load. A query with mean=1ms but called 10M times has total=10,000 seconds — far more impactful than a query with mean=10 seconds but called twice. For prioritizing optimization work, total impact (`total_exec_time`) is almost always more relevant.

Q3. How do you distinguish an I/O-bound query from a CPU-bound query?

A: In `pg_stat_statements` : I/O-bound queries have high `shared_blks_read` and `blk_read_time` . CPU-bound queries have high execution time with low disk read time. In `EXPLAIN (ANALYZE, BUFFERS)` : I/O-bound shows high `Buffers: read=N` (disk reads); CPU-bound shows high `actual time` with `Buffers: shared hit=N` (all from cache). The ratio of `blk_read_time / total_exec_time` is a good proxy: if > 50%, the query is I/O-bound; if < 10%, it's CPU-bound.

Exercises with Solutions

Exercise 1

Write a query to find all queries that:

1. Run more than 1000 times per day
2. Have mean execution time > 100ms
3. Are causing disk I/O (shared_blks_read > 0) Sort by total time impact.

Solution:

```
SELECT
    LEFT(query, 150) AS query,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec,
    shared_blks_read AS disk_reads,
    ROUND(blk_read_time::numeric, 2) AS disk_read_ms
FROM pg_stat_statements
WHERE calls > 1000
  AND mean_exec_time > 100
  AND shared_blks_read > 0
ORDER BY total_exec_time DESC
LIMIT 20;
```

Production Scenarios

Scenario 1: Unexplained 5am Performance Degradation

Users reported the application became slow every day around 5am. Investigation using pg_stat_activity and pg_stat_statements revealed:

```
-- Live query check at 5am:
SELECT pid, now()-query_start AS age, LEFT(query, 80) AS query, state, wait_event
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start;

-- Found: Several queries waiting on Lock, blocked by one long-running query
-- Long-running query: a nightly report building a temp table

-- Solution: move the nightly report to 3am, add CONCURRENTLY to index operations,
-- break the report into smaller batches to reduce lock hold time
```

Cross-References

- 01_explain_basics.md — EXPLAIN output interpretation
- 02_explain_analyze.md — EXPLAIN ANALYZE detailed analysis
- 05_query_planner.md — Planner statistics
- 06_statistics_and_estimates.md — ANALYZE
- 10_optimization_cookbook.md — Applied optimization scenarios